

EVENT-DRIVEN CSV DATA INGESTION AND GRAPH-BASED CHATBOT

Methods and Design Decisions

A modular, event-driven pipeline for CSV ingestion, graph storage, and conversational querying

Developed an Event-Driven CSV Data Ingestion and Graph-Based Chatbot that enables users to upload CRM datasets in CSV format, automatically processes and stores the data in Neo4j, and allows users to retrieve information using natural language queries. The system is built with a modular and scalable architecture.

Problem Statement

- Organizations frequently work with CSV files from CRM and business systems.
- Manual data import and querying is time-consuming.
- Traditional databases require technical SQL knowledge.
- Large datasets are difficult to analyze manually.

Project Objectives

- Automate CSV data ingestion.
- Validate and clean uploaded datasets.
- Stream data reliably using Apache Kafka.
- Store structured information in Neo4j.
- Enable chatbot-based data retrieval.
- Build a scalable and modular system.

Technology Stack

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** FastAPI
- **CSV Processing:** Pandas
- **Message Broker:** Apache Kafka
- **Database:** Neo4j
- **Query Language:** Cypher
- **Deployment:** Docker & Docker Compose

Why We Chose These Technologies

FastAPI

- High-performance REST API framework.
- Handles CSV uploads and chatbot requests.
- Easy integration with Kafka and Neo4j.

Apache Kafka

- Reliable message streaming.

- Decouples backend from database.
- Supports asynchronous processing.

Neo4j

- Graph database optimized for connected data.
- Efficient relationship traversal.
- Flexible querying using Cypher.

Pandas

- Reads CSV files efficiently.
- Supports validation and preprocessing.

Docker

- Provides consistent execution across environments.
- Simplifies deployment of multiple services.

Cypher

- Native query language for Neo4j.
- Simplifies graph data retrieval.

Key Design Decisions

- Selected an Event-Driven Architecture for scalability.
- Used Producer-Consumer communication through Kafka.
- Adopted a Graph Database instead of relational storage.
- Used MERGE to prevent duplicate graph nodes.
- Built a modular backend for easier maintenance.
- Implemented a grounded chatbot that answers only from uploaded data.

Module Overview

- Frontend
- FastAPI Backend
- Dataset Validation
- Dataset Cleaning
- Kafka Producer
- Kafka Consumer (Loader)
- Neo4j Database

- Chatbot Service
- Docker Deployment

Expected Benefits

- Automated data ingestion.
- Reduced manual effort.
- Faster data retrieval.
- Scalable architecture.
- Improved system reliability.